

FreshStock: Sistema de Gestión de Inventarios y Control de Caducidad para Pequeños Comercios de Alimentos

Brian Jesid Zambrano Poveda, Juan Pablo Cháves Estupiñán, Sadid Enrique Acosta Osorio

Resumen—El presente documento narra el proceso de diseño, modelado e implementación de *FreshStock*, una Aplicación Web Progresiva (PWA) orientada a la gestión de inventario y al control de fechas de caducidad para pequeños y medianos negocios minoristas de alimentos. El trabajo se desarrolló en tres fases a lo largo del semestre. En la primera, se analizó la problemática del sector, se diseñaron las vistas funcionales y se construyó un prototipo interactivo de alta fidelidad. En la segunda, se modeló la base de datos relacional en MySQL, definiendo la separación entre catálogo de productos y existencias físicas para habilitar el método PEPS (Primero en Entrar, Primero en Salir). En la tercera, se implementó la arquitectura del backend con Node.js y Express, se construyó un sistema de autenticación seguro basado en hashing criptográfico con bcryptjs y tokens JWT, se diseñó una capa de persistencia NoSQL con Firebase Firestore bajo la metodología *Query Driven Design*, y se implementó un proceso ETL que migra datos desde Firestore hacia MySQL. El resultado es un sistema funcional que integra frontend, backend, base de datos relacional, autenticación por roles y analítica NoSQL.

Index Terms—Gestión de inventario, control de caducidad, PWA, método PEPS, modelo Entidad-Relación, base de datos relacional, hashing criptográfico, autenticación JWT, NoSQL, Firebase Firestore, ETL, Query Driven Design, trazabilidad, arquitectura de software.

I. PLANTEAMIENTO DEL PROBLEMA

TODO comenzó con una pregunta simple: cómo puede una frutería de barrio saber cuáles de sus yogures vencen mañana sin tener que revisar cada estante uno por uno? La respuesta, por sorprendente que parezca, es que en la mayoría de los pequeños comercios de alimentos, esa pregunta no tiene solución. Los inventarios se llevan en cuadernos, las fechas de caducidad se memorizan y las pérdidas se descubren cuando ya es demasiado tarde. Este documento cuenta cómo un equipo de tres estudiantes de Ingeniería Multimedia decidió construir una herramienta para cambiar esa realidad [?].

I-A. Situación Actual y Problemática

La problemática central radica en la **falta de trazabilidad y control en tiempo real sobre los productos perecederos**. Las principales dificultades identificadas son:

- **Desconocimiento del stock real:** la disponibilidad exacta solo se confirma mediante verificación física, generando interrupciones en el servicio.
- **Gestión ineficiente de caducidades:** los productos se organizan por orden de llegada, permitiendo que los artículos más antiguos permanezcan al fondo y caduquen sin ser rotados oportunamente [?].

- **Reabastecimiento reactivo:** las compras se realizan únicamente cuando la escasez se detecta visualmente, imposibilitando la planificación anticipada.

I-B. Perfiles de Usuario Objetivo

El sistema está dirigido a tres perfiles operativos diferenciados:

- **Administradores:** requieren visibilidad global, reportes de merma y control de usuarios del sistema.
- **Encargados de almacén:** responsables del registro de entradas y de la gestión de lotes por fecha de caducidad.
- **Vendedores:** necesitan registrar salidas y consultar disponibilidad de forma ágil durante la atención al cliente.

I-C. Consecuencias de No Resolver el Problema

La gestión actual genera impactos directos sobre la viabilidad del negocio, resumidos en la Tabla ??.

Tabla I
IMPACTO NEGATIVO DE LA GESTIÓN MANUAL DE INVENTARIOS

Impacto	Descripción
Pérdidas económicas	Desechar productos por caducidad representa una pérdida directa e irreparable de capital.
Daño reputacional	Vender productos próximos a vencer genera inconformidad en el cliente y deteriora la imagen del negocio.
Ineficiencia operativa	El personal invierte tiempo en conteos manuales en lugar de dedicarlo a la atención al cliente.

II. JUSTIFICACIÓN DE LA SOLUCIÓN

II-A. El Impacto del Desperdicio y la Necesidad de Control

Antes de escribir una sola línea de código, el equipo necesitaba entender si el problema valía la pena resolver. Los datos fueron contundentes: a nivel global, la ineficiencia en la cadena de suministro y la mala gestión de inventarios de perecederos contribuyen significativamente a las pérdidas económicas y al desperdicio de alimentos [?]. Un estudio sobre prácticas de gestión en cadenas de suministro de alimentos confirmó que los comercios que no emplean sistemas de control estandarizados pierden entre un 15 % y un 30 % de su inventario por caducidad [?].

En el contexto de los pequeños comercios, implementar prácticas estandarizadas de gestión de inventario constituye un factor crítico de éxito que con frecuencia se descuida por limitaciones técnicas y presupuestales [?]. *FreshStock* se propone cerrar esta brecha mediante la democratización del acceso a herramientas de control avanzado, transformando un modelo *reactivo* en uno *proactivo*.

II-B. Viabilidad Tecnológica: PWA

La decisión de construir una **Aplicación Web Progresiva (PWA)** no fue arbitraria. El equipo evaluó tres opciones: una app nativa para Android/iOS, una app de escritorio, y una PWA. Las dos primeras requerían desarrollo separado por plataforma y publicación en tiendas, lo que multiplicaba el costo y el tiempo. La PWA, en cambio, ofrece dos ventajas decisivas para el contexto del proyecto:

- **Despliegue de bajo costo:** opera desde los *smartphones* o computadores ya disponibles en el negocio, sin hardware especializado.
- **Sincronización y concurrencia:** múltiples usuarios registran entradas y salidas simultáneamente, manteniendo una única fuente de verdad centralizada.

Además, las PWAs pueden funcionar *offline* y actualizarse automáticamente, dos características esenciales para un negocio donde la conexión a internet no siempre es estable.

II-C. Matriz de Operaciones CRUD

Para garantizar la inmutabilidad de los historiales, el equipo definió desde el inicio qué operaciones estarían permitidas y cuáles restringidas. Esta decisión se tomó antes de escribir cualquier código, como guía para el modelado de la base de datos que se desarrollaría en el segundo corte. La Tabla ?? resume las decisiones.

Tabla II
OPERACIONES CRUD Y RESTRICCIONES POR ENTIDAD

Entidad	Crear	Leer	Actualizar	Eliminar
Usuarios	Sí	Sí	Sí	Sí
Productos	Sí	Sí	Sí	Sí
Lotes	Sí	Sí	Sí	No*
Movimientos	Sí	Sí	No*	No*

*Restringido para preservar la integridad de auditoría.

III. ESTADO DEL ARTE

Antes de diseñar *FreshStock*, el equipo dedicó tiempo a entender qué soluciones ya existían en el mercado. La pregunta no era solo “¿qué herramientas hay disponibles?”, sino “¿por qué ninguna de ellas resuelve el problema de una frutería pequeña?” [?].

III-A. Análisis de Soluciones Existentes

Se analizaron tres referentes representativos del mercado:

- **Sortly:** aplicación de gestión visual intuitiva; carece de funcionalidades para control de caducidades y método **PEPS**. Presenta costo elevado para el segmento objetivo.
- **Odoo Inventory:** ERP robusto con trazabilidad completa, cuya curva de aprendizaje resulta excesiva para micronegocios sin personal técnico dedicado.
- **Inventory Now:** solución simple con soporte *offline*, pero sin gestión de fechas de vencimiento ni soporte para múltiples sucursales.

III-B. Diferenciación de FreshStock

Tras analizar los tres referentes, el equipo llegó a una conclusión clara: ninguna herramienta existente resolvía el problema específico de los perecederos de forma nativa. *FreshStock* se distingue por su **enfoque nativo en caducidades y en el método PEPS**. Integra alertas proactivas diseñadas específicamente para productos perecederos, combinando simplicidad de uso con trazabilidad avanzada a un costo significativamente menor que los ERPs disponibles en el mercado.

IV. DEFINICIÓN FUNCIONAL DEL SISTEMA

Con los referentes analizados y la diferenciación clara, el equipo se sentó a definir qué haría *FreshStock* y qué no. No se trataba de construir un ERP genérico, sino de diseñar un sistema que resolviera los tres problemas identificados en la sección anterior: desconocimiento del stock, gestión ineficiente de caducidades y reabastecimiento reactivo [?].

IV-A. Módulos Centrales de Operación

El sistema se divide en cuatro ejes funcionales interconectados, cada uno diseñado para atacar una faceta específica del problema:

- **Dashboard de control:** panel analítico que consolida indicadores clave de rendimiento (KPI) y centraliza alertas críticas en tiempo real. Responde al problema del desconocimiento del stock.
- **Catálogo maestro:** módulo de definición de artículos donde se establecen parámetros base para automatizar alertas de reabastecimiento.
- **Gestión de lotes y método PEPS:** agrupa las entradas en lotes y aplica estrictamente el método **Primero en Entrar, Primero en Salir** [?], descontando automáticamente el lote con fecha de caducidad más próxima en cada proceso de salida o venta.
- **Motor de trazabilidad:** registro inmutable de cada transacción etiquetada con marca de tiempo y autor de la acción.

IV-B. Flujo Lógico y Ciclo de Vida del Producto

El ciclo de vida de la información opera en tres fases, ilustradas en la Fig. ?. Estas fases reflejan cómo un producto físico —un yogur, una manzana, un litro de leche— se convierte en datos dentro del sistema y eventualmente en una alerta o una venta:

1. **Registro (entrada):** ingreso de nueva mercancía con asignación obligatoria de fecha de vencimiento.
2. **Monitoreo activo:** un *cron job* evalúa diariamente cada lote, cambiando su estado de *Normal* a *Riesgo* y luego a *Vencido*.
3. **Ejecución (salida):** al registrar una venta, la lógica de base de datos subtrae del lote más antiguo, garantizando la rotación **PEPS**.



Figura 1. Flujo lógico de información y aplicación del método **PEPS**.

V. FLUJO DE NAVEGACIÓN

Definido qué haría el sistema, el siguiente paso fue diseñar cómo se movería el usuario entre pantallas. Una cosa es tener un dashboard con KPIs y otra muy distinta es lograr que un vendedor, sin entrenamiento técnico, pueda registrar una venta en menos de tres toques. El equipo dibujó el mapa de navegación antes de escribir cualquier componente de React o línea de JavaScript [?].

V-A. Mapa Estructural de Vistas

El mapa de navegación garantiza acceso rápido a las funciones operativas desde cualquier nivel de la jerarquía de pantallas. La estructura de vistas se organiza así:

```

LOGIN
+-- DASHBOARD (Alertas y métricas)
+-- CATALOGO (Búsqueda / creación)
+-- LOTES (Entradas y mermas)
+-- PROVEEDORES (Contactos y precios)
+-- REPORTE (Auditoría)
+-- USUARIOS (Roles y permisos)

```

Esta estructura de dos niveles se eligió deliberadamente: el usuario siempre sabe dónde está (el dashboard como centro) y cómo llegar a cualquier función en un solo clic.

V-B. Identificación de Acciones del Usuario

La Tabla ?? detalla las operaciones críticas que el usuario ejecuta en cada vista para mantener activo el modelo **PEPS** y el control de caducidades.

Tabla III
ACCIONES OPERATIVAS Y FUNCIONALES POR VISTA

Vista	Acciones clave del usuario
Dashboard	Revisar métricas generales y acceder mediante atajos a productos próximos a caducar.
Alertas	Gestionar notificaciones críticas y ejecutar acciones como aplicar descuentos o registrar mermas.
Lotes	Visualizar el orden de salida PEPS e ingresar mercancía con fecha de vencimiento.
Catálogo	Consultar inventario e identificar visualmente el estado del stock (normal, bajo o crítico).

VI. DESARROLLO DE LA INTERFAZ Y PROTOTIPO

Con el flujo de navegación definido, el equipo pasó a construir la interfaz. No se trataba de hacer algo bonito por hacerlo: en un almacén donde varios vendedores atienden clientes al mismo tiempo, una interfaz confusa puede significar que se registre el lote equivocado o que un producto venido se venda por error. El diseño debía ser funcional antes que estético [?].

VI-A. Diseño Centrado en el Usuario y Semántica Visual

Se desarrolló un prototipo navegable de alta fidelidad basado en los lineamientos de *Material Design 3* [?]. El objetivo fue minimizar la carga cognitiva del usuario en entornos de alta rotación comercial. Se implementó un sistema de **semántica de colores** estandarizado para guiar decisiones operativas en tiempo real:

- **Rojo (crítico):** acciones destructivas, productos vencidos o quiebre de stock.
- **Amarillo / Naranja (advertencia):** lotes que entrarán en ventana de caducidad en los próximos 7 días.
- **Verde / Neutro (óptimo):** estados normales de inventario y retroalimentación positiva ante acciones completadas.

Esta coherencia visual guía la atención del operario hacia las tareas prioritarias, atacando directamente la problemática descrita en la Sección ??.

VI-B. Vistas Funcionales del Sistema

Con el sistema de colores definido, el equipo construyó las vistas funcionales. Cada pantalla se diseñó pensando en un escenario real del negocio: un vendedor consultando stock, un encargado registrando un lote, un administrador revisando alertas. A continuación se presentan las vistas que conforman el ecosistema de la *PWA FreshStock*.

VI-B1. Vista General y Alertas Tempranas: El **Dashboard** (Fig. ??) funciona como centro de mando: prioriza las alertas del día y grafica el flujo de entradas y salidas, otorgando al administrador visibilidad instantánea del estado operativo. El **Centro de Alertas** (Fig. ??) permite ejecutar acciones directas desde la notificación —como dar de baja un producto o aplicar promociones para evitar mermas—.

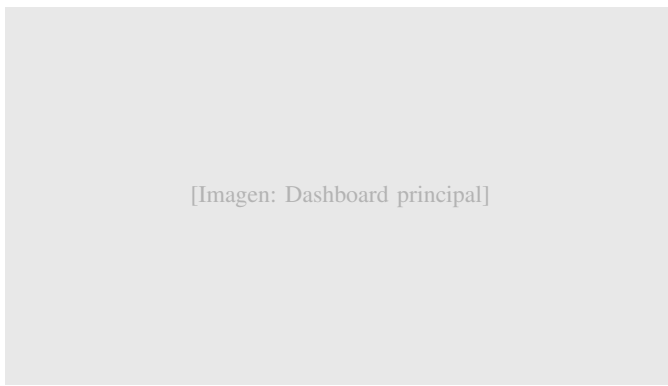


Figura 2. Dashboard principal con resumen métrico y atajos de alerta.



Figura 3. Centro de alertas clasificado por nivel de criticidad.

*VI-B2. Control **PEPS** y Catálogo Visual:* La vista de **Gestión de Lotes** (Fig. ??) organiza la mercancía automáticamente por fecha de caducidad e indica qué lote debe salir primero según la prioridad **PEPS**. El **Catálogo de Productos** (Fig. ??) utiliza tarjetas visuales para identificar rápidamente la disponibilidad y el umbral de stock de cada artículo.



Figura 4. Gestión de lotes con priorización automática del método **PEPS**.

VI-B3. Auditoría, Proveedores y Administración: El módulo de **Reportes** (Fig. ??) documenta cada movimiento de forma inmutable, destacando los productos con mayores pérdidas. El módulo de **Proveedores** (Fig. ??) centraliza contactos para facilitar el reabastecimiento. La gestión de **Usuarios** (Fig. ??) restringe las acciones críticas al rol de administrador.



Figura 5. Catálogo de productos con indicadores visuales de stock.



Figura 6. Panel de reportes con historial de auditoría inmutable.



Figura 7. Directorio y métricas de rendimiento de proveedores.



Figura 8. Gestión de usuarios y matriz de permisos por roles.

VII. MODELADO E IMPLEMENTACIÓN DE LA BASE DE DATOS

Con la interfaz diseñada y el flujo de navegación definido, el equipo enfrentó la siguiente pregunta: ¿cómo se organiza la información detrás de las pantallas? Un dashboard bonito es inútil si la base de datos no puede responder rápidamente cuáles lotes vencen esta semana. Este segundo corte se dedicó enteramente a diseñar la estructura relacional que soportaría toda la lógica de negocio [?].

VII-A. Modelo Entidad-Relación

El Diagrama Entidad-Relación (Fig. ??) fue el punto de partida. Antes de crear tablas en MySQL, el equipo dibujó en papel cómo se relacionaban las entidades centrales: productos, lotes, movimientos, proveedores y usuarios. Cada caja representa una tabla; las líneas que las conectan indicaron que comparten información, y los símbolos en sus extremos expresaron cuántos registros de un lado pueden relacionarse con cuántos del otro (cardinalidad).



[Imagen: Diagrama ER FreshStock]

Figura 9. Diagrama Entidad-Relación de *FreshStock*. Cada caja es una tabla; las líneas expresan relaciones y cardinalidades.

VII-B. Justificación del Diseño

Con la estructura general en mente, se explican a continuación las decisiones de modelado más relevantes.

VII-B1. Separación entre Catálogo y Existencias Físicas:

La decisión más trascendental fue separar el catálogo maestro (PRODUCTS) de las existencias físicas (BATCHES). Un mismo producto —por ejemplo, “Yogurt”— puede llegar al almacén en distintas fechas y con distintas fechas de vencimiento. Sin esta separación, sería imposible identificar qué unidades vencerán primero. Al agrupar en lotes, el sistema identifica automáticamente cuál debe salir antes, ejecutando el método **PEPS** con exactitud [?].

La entidad **MOVEMENTS** actúa como libro contable digital: registra de forma inmutable cada entrada, salida o merma, junto con el usuario responsable y la marca de tiempo exacta.

VII-B2. Relación N:M entre Productos y Proveedores:

Un proveedor suministra múltiples productos; un producto puede adquirirse a varios proveedores. Esta relación de muchos a muchos (N:M) se resuelve con la tabla asociativa **PRODUCT_SUPPLIERS**, que además almacena el atributo **purchase_price**: el precio exacto al que cada proveedor vende cada producto, información esencial para comparar costos y calcular márgenes de rentabilidad.

VII-B3. Integridad Referencial y Tipos de Datos: Se configuró integridad referencial activa: **ON DELETE RESTRICT** impide borrar un producto con lotes asociados, y **ON DELETE CASCADE** elimina registros dependientes cuando corresponde. Se emplearon tipos especializados —**DECIMAL** para montos monetarios y **TIMESTAMP** para auditoría con marca de tiempo precisa— para garantizar exactitud en todos los registros operativos [?].

VIII. ARQUITECTURA DEL BACKEND Y LÓGICA DE SERVIDOR

VIII-A. Decisiones de plataforma: por qué Node.js y Express

Cuando el equipo comenzó a construir el servidor de *FreshStock*, la primera duda fue qué tecnología usar en el backend. No se trataba de un sistema masivo, sino de una frutería donde varios vendedores consultan stock al mismo tiempo. La clave era que el servidor no se bloqueara esperando respuestas de la base de datos.

Se eligió **Node.js** porque trabaja con un modelo *event-driven*: en lugar de crear un hilo nuevo por cada petición, usa un único hilo con un *event loop* que va atendiendo las solicitudes a medida que llegan. Esto es muy útil cuando las operaciones son principalmente consultas a una base de datos, porque no se desperdicia memoria creando procesos innecesarios.

Sobre Node se instaló **Express.js** [?], un framework que simplifica mucho la creación de APIs siguiendo los principios de la arquitectura REST [?]. Express se encarga de rutas, cabeceras HTTP y parsing de JSON, así que el equipo pudo concentrarse en la lógica del inventario en lugar de programar un servidor desde cero. Además, su sistema de middleware permitió agregar después el manejo de errores y CORS sin modificar los endpoints directamente.

VIII-B. Separando responsabilidades: Routes, Controllers y DAOs

En las primeras versiones del código, todo estaba mezclado: los mismos archivos que consultaban la base de datos también recibían las peticiones HTTP y devolvían respuestas JSON. Era difícil de mantener porque cualquier cambio en una parte rompía otra. Ahí se entendió la importancia del principio de responsabilidad única (**SRP**).

Se decidió dividir el backend en tres capas bien definidas:

1. **Routes:** se encargan solo del enrutamiento. Definen qué URL lleva a qué función, sin meterse en lógica de negocio. Ejemplo: `router.get('/products', productController.getAll)`.
2. **Controllers:** actúan como intermediarios. Reciben la petición del cliente, extraen los datos que necesitan, llaman

a los DAOs y devuelven la respuesta en JSON. No saben nada de SQL.

3. **DAOs (Data Access Objects):** son los que hablan directamente con MySQL. Ejecutan las consultas y devuelven los resultados. No saben que existe HTTP.

Esta separación tiene una ventaja práctica muy clara: si algún día se quiere cambiar de MySQL a PostgreSQL, solo hay que reescribir los DAOs. Los controllers y las rutas quedan intactos.

VIII-C. *asyncHandler, AppError y el pool de conexiones*

Un problema recurrente era repetir el mismo bloque `try/catch` en cada controller. Se hacía tedioso y facilitaba errores por descuido. La solución fue crear `asyncHandler`: una función de orden superior que envuelve los controllers, atrapa cualquier error de una promesa y lo envía automáticamente al middleware de errores de Express mediante `next(error)`.

Pero atrapar errores no era suficiente; también había que distinguir qué tipo de error era. Responder siempre “Error interno del servidor” con código 500 no ayudaba al frontend ni al programador. Se creó entonces la clase **AppError**, que extiende la clase nativa `Error` y le agrega un código HTTP propio: 404 cuando no se encuentra un recurso, 400 cuando los datos son inválidos, 409 cuando hay un duplicado. El middleware de errores detecta si el error es instancia de `AppError` y responde con el código correcto, sin necesidad de poner `if` en cada controller.

Por último, en la base de datos se evitó abrir una conexión nueva por cada consulta. Eso es ineficiente porque cada conexión nueva implica un *handshake* con el motor. En su lugar, se configuró un **pool de conexiones** en `mysql.service.js` usando `mysql2/promise`. El pool mantiene varias conexiones abiertas desde el inicio y las reutiliza. Así, cuando tres vendedores consultan stock al mismo tiempo, el servidor usa conexiones que ya están listas, sin saturar MySQL.

VIII-D. *Cómo viaja una petición: ejemplo práctico*

Para entender cómo funciona todo junto, conviene ver el recorrido de una petición real. Supongamos que el encargado del almacén abre la app y toca “Ver lotes próximos a vencer”. Su navegador envía una petición `GET /api/batches/expiring`.

Express recibe esa petición, pasa por el middleware de CORS y JSON parsing, y luego la ruta `batches.routes.js` la deriva al controller correspondiente. El controller, protegido por `asyncHandler`, extrae el parámetro `days` de la query string, lo revisa y llama a `batchDao.getExpiringBatches(days)`.

El DAO arma la consulta SQL usando `JOINS` entre las tablas `batch` y `product`, y aplica la función `DATEDIFF` de MySQL para filtrar solo los lotes que vencen dentro de los próximos días. Esa consulta se ejecuta a través del pool de conexiones. MySQL devuelve las filas, el DAO las pasa al controller, y el controller las envía como JSON con código 200 OK.

Si en algún punto falla algo —por ejemplo, la base de datos está caída—, `asyncHandler` captura el error y lo

manda al middleware de errores. Como no es un `AppError`, el middleware responde 500 Internal Server Error. Lo importante es que el flujo sigue siendo limpio: el DAO nunca toca `res`, el controller nunca ejecuta SQL, y la ruta nunca tiene lógica de negocio.

Así se logró una arquitectura donde cada capa tiene una única tarea. No es la solución más sofisticada del mundo, pero cumple con lo que necesita *FreshStock*: que sea entendible, mantenible y que no se rompa cuando hay varios usuarios al mismo tiempo.

IX. IMPLEMENTACIÓN DE LOGIN SEGURO CON HASHING CRIPTOGRÁFICO

IX-A. *El problema de las contraseñas en texto plano*

Cuando el equipo revisó el estado inicial del sistema, se encontró con que la tabla `USER` de la base de datos ya tenía definida una columna `password_hash` de tipo `VARCHAR(255)`, pero el código del backend no hacía nada con ella. El formulario de login del frontend simplemente esperaba 800 milisegundos con un spinner y luego navegaba al sin importar las credenciales ingresadas. No existía ningún endpoint de autenticación, no se verificaban contraseñas y no se generaban tokens de sesión.

Esto representaba un riesgo crítico: si alguien accedía a la base de datos, podía leer todas las contraseñas de los usuarios directamente. El equipo necesitaba un sistema donde la contraseña original nunca se almacenara, y donde cada inicio de sesión se validara de forma segura.

IX-B. *Selección del algoritmo: por qué bcryptjs*

El equipo evaluó tres opciones para el hashing de contraseñas: **argon2**, considerada la más segura en la actualidad pero con dependencias nativas que complicaban la instalación en Windows; **bcrypt**, muy probada en producción y recomendada por el NIST como función de hashing para contraseñas [?], pero que requería compilación de módulos C++ que generaba errores frecuentes en distintos sistemas operativos; y **bcryptjs** [?], una reimplementación pura en JavaScript que produce resultados idénticos a `bcrypt` sin necesidad de compilación alguna. Además, las guías de seguridad de OWASP confirman que `bcrypt` cumple con las mejores prácticas de almacenamiento de contraseñas [?].

Se eligió **bcryptjs** por tres razones concretas. Primera, al ser JavaScript puro, funcionaba sin problemas en cualquier máquina donde corriera Node.js, sin importar el sistema operativo. Segunda, producía hashes compatibles con el formato `$2b$` de `bcrypt`, lo cual facilitaba una futura migración a la librería nativa si el rendimiento lo exigía. Tercera, ya estaba incluida en el `package.json` del proyecto desde la planificación inicial, así que no fue necesario instalar nada adicional.

IX-C. *Cómo funciona el algoritmo de hashing*

El algoritmo implementado sigue el siguiente flujo. Cuando un usuario se registra o cambia su contraseña, la función `hashService.hash(plainText)` genera un **salt** aleatorio de 16 bytes usando el generador criptográfico del sistema

operativo (`crypto.randomBytes`). Luego, toma la contraseña en texto plano y el salt, y ejecuta el algoritmo Blowfish modificado (Eksblowfish) durante un número configurable de rondas —en este caso, 10—. Cada ronda aplica una serie de operaciones XOR y sustituciones a través de cajas S, lo que genera una carga computacional intencional. El resultado final es un string con el siguiente formato:

```
$2b$10$A8XvRgy2FFIVD1O.rKDY8.tJN8r1...
```

Donde 2b identifica la versión del algoritmo, 10 indica el número de rondas (costo), los 22 caracteres siguientes representan el salt en Base64, y el resto es el hash resultante. Lo importante es que el salt está embebido dentro del mismo string: no es necesario almacenarlo por separado.

Para verificar una contraseña, la función `hashService.compare(plainText, hashedValue)` extrae el salt del hash almacenado, vuelve a ejecutar Blowfish con ese mismo salt y la contraseña ingresada, y compara el resultado. Si coinciden, la contraseña es válida.

IX-D. Justificación técnica de las decisiones

La decisión de usar 10 rondas de costo no fue arbitraria. Con 10 rondas, el hashing tarda aproximadamente 100 milisegundos en un servidor convencional. Esto es lo suficientemente rápido para que un usuario no note el retraso durante el login, pero lo suficientemente lento como para que un atacante con hardware especializado no pueda probar miles de contraseñas por segundo. Si en el futuro el hardware se vuelve más rápido, basta con cambiar el número de rondas a 12 o 14 —los hashes viejos siguen funcionando porque el costo está almacenado dentro del propio hash—.

El uso de **JWT (JSON Web Tokens)** para la sesión también fue una decisión deliberada. A diferencia de las sesiones tradicionales con cookies del lado del servidor, JWT permite que el token se almacene en el `localStorage` del navegador y se envíe como cabecera en cada petición. Esto evita necesitar una tabla de sesiones en la base de datos y simplifica la arquitectura. El token incluye el ID del usuario, su correo y su rol, y tiene un tiempo de expiración de 8 horas.

Por último, se creó un servicio aislado (`hash.service.js`) que encapsula toda la lógica de hashing. Los controllers nunca llaman directamente a `bcryptjs`; llaman a este servicio. Esto cumple con el principio de responsabilidad única: si algún día se quiere cambiar a `argon2`, solo se modifica un archivo.

IX-E. Flujo completo de autenticación

El flujo de registro funciona así. El usuario llena el formulario en la vista `screen-register` con su nombre, correo, contraseña y rol. El frontend envía una petición `POST /api/auth/register` al servidor. El controller de autenticación valida que no falten campos, que la contraseña tenga al menos 6 caracteres y que el correo no esté registrado. Si todo está bien, genera el hash de la contraseña, inserta el

usuario en la tabla `USER` y devuelve un JWT junto con los datos del usuario.

El flujo de inicio de sesión es similar. El usuario ingresa su correo y contraseña en `screen-login`. El frontend envía `POST /api/auth/login`. El DAO busca el usuario por correo —incluyendo el `password_hash` que normalmente se excluye en las consultas—. El controller verifica la contraseña contra el hash almacenado usando `hashService.compare`. Si coincide y el usuario está activo, devuelve el JWT. Si no coincide, responde con 401 Credenciales inválidas, sin revelar si el problema es el correo o la contraseña —una medida básica contra la enumeración de usuarios—.

En el frontend, el token y los datos del usuario se almacenan en `localStorage`. Cada vez que la página se recarga, el estado de la aplicación (`AppState`) recupera estos valores y, si existen, redirige automáticamente al dashboard sin pedir login nuevamente. El botón de cerrar sesión limpia el `localStorage` y devuelve al usuario a la pantalla de login. Además, el router del frontend protege las vistas: si un usuario sin token intenta acceder a cualquier pantalla que no sea login o registro, lo redirige automáticamente al login.

De esta manera, el equipo logró un sistema de autenticación completo donde la contraseña original nunca sale del navegador del usuario, se almacena únicamente como hash irreversible en MySQL, y cada sesión se gestiona con tokens firmados criptográficamente.

IX-F. Control de acceso basado en roles

Una vez que el sistema de autenticación estaba funcionando, el equipo identificó un problema inmediato: cualquier usuario que iniciara sesión podía acceder a la información de todos los demás. La pantalla de usuarios mostraba la lista completa sin importar quién estuviera conectado. En un negocio real, eso significaba que un vendedor podía ver los datos del administrador, sus correos y sus roles. No había ningún tipo de restricción.

Para resolverlo, se creó un middleware de autenticación ubicado en `utils/auth.middleware.js`. Su trabajo es simple pero esencial: antes de que la petición llegue al controller, el middleware extrae el token del header `Authorization`, lo decodifica con `jwt.verify()` y adjunta los datos del usuario (`id`, `email`, `role`) al objeto `req`. Si el token no existe o está expirado, la petición se detiene inmediatamente con un error 401.

Con el usuario identificado, se modificó el controller de usuarios para aplicar una regla de negocio basada en roles. Si el rol del usuario es `admin`, el endpoint ejecuta un `SELECT` que devuelve todos los registros de la tabla `USER`. Si el rol es cualquier otro (`warehouse` o `seller`), el endpoint filtra por el `user_id` extraído del token, devolviendo únicamente el registro del propio usuario. Así, un vendedor que entre a la sección de usuarios solo se ve a sí mismo, mientras que el administrador tiene visibilidad completa del equipo.

Esta decisión se fundamentó en el principio de **mínimo privilegio**: cada usuario debe tener acceso solo a la información que necesita para cumplir su función. Además, la lógica de autorización quedó aislada en el middleware, separada del

controller y del DAO, cumpliendo así con el principio de responsabilidad única.

IX-G. Personalización dinámica de la interfaz

Durante las pruebas de integración, el equipo detectó un detalle que parecía menor pero que rompía la experiencia del usuario: el mensaje de bienvenida del dashboard mostraba de forma estática el texto “Buenos días, Sadid!”, sin importar quién hubiera iniciado sesión. Esto ocurrió porque el nombre estaba escrito directamente en el archivo HTML, sin ninguna vinculación con los datos de la sesión activa.

La solución fue conectar el texto de bienvenida con el estado de la aplicación. Cuando un usuario inicia sesión, el frontend almacena sus datos —nombre, correo y rol— en el objeto global `AppState` y en el `localStorage` del navegador [?]. Al cargar el dashboard, una instrucción de JavaScript reemplaza el texto estático por el valor de `AppState.user.name`, mostrando así un saludo personalizado.

Este mismo mecanismo se aplica a los paneles laterales de navegación. La función `updateSidebarUser()` actualiza dinámicamente el avatar con las iniciales del usuario, su nombre completo y su rol traducido (“Administrador”, “Almacén” o “Vendedor”). Esto garantiza que, sin importar en qué pantalla se encuentre, el usuario siempre vea su propia identidad reflejada en la interfaz.

La persistencia de estos datos en `localStorage` también resolvió otro problema: al recargar la página, el sistema recupera el token y los datos del usuario, redirigiéndolo automáticamente al dashboard sin necesidad de volver a iniciar sesión. Solo cuando el token expira o el usuario cierra sesión explícitamente se borran los datos locales y se regresa al login.

X. DISEÑO E IMPLEMENTACIÓN NOSQL

El sistema, hasta este punto, funcionaba con una sola base de datos relacional en MySQL. Sin embargo, el equipo identificó una limitación importante: los datos operativos (movimientos, lotes, alertas) estaban optimizados para consultas transaccionales, pero no para análisis de tendencias ni para procesamiento masivo. Para habilitar capacidades de Big Data y analítica en tiempo real, se diseñó una capa de persistencia NoSQL complementaria utilizando Firebase Firestore [?].

X-A. Metodología Query Driven Design

La metodología *Query Driven Design* [?] establece que la estructura de una base de datos NoSQL debe diseñarse a partir de las consultas que el sistema necesitará responder, y no a partir de las entidades del mundo real como en el modelado relacional. El equipo definió cinco preguntas de negocio antes de escribir ninguna línea de código:

1. ¿Cuáles son los productos más vendidos en los últimos 7 días?
2. ¿Cuánto inventario se desperdició por caducidad este mes?
3. ¿Cuál es la duración promedio de las sesiones de los vendedores?
4. ¿Qué productos se consultan con más frecuencia pero no se compran?

5. ¿Cómo evoluciona el stock de un producto a lo largo del tiempo?

Con estas preguntas como guía, se diseñó una única colección en Firestore llamada `analytics_events`, donde cada documento representa un evento del sistema.

X-B. Justificación: datos embebidos vs. referenciados

En Firestore existen dos estrategias para organizar datos relacionados [?]: **embebidos** (datos anidados dentro del mismo documento) y **referenciados** (datos en colecciones separadas vinculados por ID).

El equipo eligió la estrategia **embebida** por tres razones:

- **Consultas eficientes:** al embeber los datos del evento dentro del documento, una sola lectura devuelve toda la información necesaria, sin necesidad de hacer *joins* entre colecciones.
- **Escritura atómica:** cada evento se almacena como un documento independiente, lo que permite escrituras concurrentes sin bloqueos.
- **Escalabilidad:** Firestore escala automáticamente con el volumen de escrituras, y al usar documentos embebidos se minimiza el número de operaciones de lectura.

La estructura de cada documento es la siguiente:

```
{
  event_type: "inventory_movement",
  timestamp: Timestamp,
  user_id: 1,
  data: {
    product_sku: "YOGURT001",
    batch_id: 5,
    movement_type: "purchase",
    quantity: 50,
    previous_quantity: 0,
    posterior_quantity: 50
  },
  metadata: {
    source: "web",
    app_version: "1.0"
  }
}
```

X-C. Estructura de la colección

La colección `analytics_events` almacena cinco tipos de eventos, cada uno con su propio esquema dentro del campo `data`:

- **inventory_movement:** registra cada entrada o salida de inventario con SKU, lote, cantidad y tipo de movimiento.
- **stock_snapshot:** captura el estado del stock en un momento dado, habilitando series de tiempo.
- **user_session:** registra cada inicio de sesión con usuario, IP y dispositivo.
- **expiry_event:** documenta cuando un lote vence o se elimina, indicando si fue consumido o desperdiciado.
- **product_view:** rastrea qué productos consulta cada usuario, para análisis de comportamiento.

X-D. Integración con Firebase Firestore

La integración se implementó mediante el SDK `firebase-admin` en el backend. El servicio `firebase.service.js` inicializa la conexión utilizando una clave de servicio almacenada localmente, y el servicio `analytics.service.js` expone funciones específicas para cada tipo de evento. Los eventos se envían de forma **asíncrona y no bloqueante**: la operación principal (crear lote, iniciar sesión, etc.) se completa primero, y el evento a Firebase se envía en segundo plano sin afectar la experiencia del usuario.

XI. PROCESO ETL

Con los datos recolectándose en Firebase, el siguiente paso fue construir un proceso ETL (*Extract, Transform, Load*) [?] que permitiera migrar esos datos a la base de datos relacional de MySQL, donde pueden integrarse con los datos operativos existentes para generar reportes consolidados [?].

XI-A. Arquitectura del proceso

El proceso ETL de *FreshStock* opera en tres fases:

1. **Extract:** el controller de ETL consulta todos los documentos de la colección `analytics_events` en Firestore, ordenados por fecha descendente.
2. **Transform:** cada evento se mapea a la estructura de las tablas de MySQL según su tipo: `expiry_event` → `alert`, `inventory_movement` → `movement`, `user_session` → `audit_session`.
3. **Load:** los datos transformados se insertan en las tablas correspondientes, verificando duplicados antes de cada inserción.

XI-B. Mapeo de datos NoSQL a MySQL

El proceso de transformación aplica las siguientes reglas de mapeo:

- **expiry_event** → **alert:** el campo `product_name` se convierte en el título de la alerta. El campo `was_consumed` determina el tipo de alerta: si fue desperdiciado, se asigna `alert_type_id = 2` (expired); si fue consumido, `alert_type_id = 4` (batch_depleted).
- **inventory_movement** → **movement:** los campos `quantity`, `previous_quantity` y `posterior_quantity` se transfieren directamente. El tipo de movimiento se mapea: `purchase` → `movement_type_id = 1`, `sale` → `movement_type_id = 2`.
- **user_session** → **audit_session:** se crea un registro con el `user_id`, la IP y el user agent del evento. El campo `token_hash` se marca como `etl-import` para diferenciarlo de las sesiones creadas en tiempo real.

XI-C. Integración frontend-backend

El proceso ETL se activa mediante un botón en el Dashboard del sistema, visible exclusivamente para usuarios con rol de administrador. Al hacer clic, el frontend envía una petición

`POST /api/etl/run` al servidor, que ejecuta el proceso completo y devuelve un resumen con la cantidad de eventos procesados, insertados y omitidos. El botón muestra un spinner durante la ejecución y, al finalizar, presenta un *toast* de notificación con los resultados.

Esta integración garantiza que cualquier administrador pueda sincronizar los datos entre Firebase y MySQL con un solo clic, sin necesidad de acceder al servidor ni ejecutar comandos manualmente.

XI-D. Resultados de la migración

En las pruebas realizadas, el proceso ETL demostró que los datos migran correctamente desde Firestore hacia MySQL. Se verificó que los eventos de tipo `inventory_movement` se insertan en la tabla `movement`, que los eventos `user_session` generan registros en `audit_session`, y que los eventos `expiry_event` crean alertas en la tabla `alert` —una tabla que, hasta este punto del proyecto, permanecía vacía a pesar de estar definida en el esquema desde el segundo corte—.

XII. CONCLUSIÓN

El desarrollo de *FreshStock* a lo largo de los tres cortes del semestre demostró que un equipo pequeño, con las herramientas correctas, puede construir una solución funcional para un problema real. El proyecto partió de una problemática concreta —las pérdidas económicas por caducidad en pequeños comercios de alimentos— y la abordó de forma progresiva, capa por capa, hasta conseguir un sistema completo que integra frontend, backend, base de datos relacional, autenticación segura y persistencia NoSQL.

En el primer corte, se analizó el contexto del sector, se definieron los perfiles de usuario y se diseñaron las interfaces basadas en Material Design 3. El prototipo resultante estableció la base visual y funcional sobre la que se construiría el resto del sistema. En el segundo corte, se modeló la base de datos relacional en MySQL, tomando la decisión estratégica de separar el catálogo de productos de las existencias físicas para habilitar el método PEPS y la trazabilidad inmutable [?].

En el tercer corte, el trabajo se centró en tres pilares. Primero, se construyó la arquitectura del backend con Node.js y Express [?], organizando el código en tres capas (routes, controllers, DAOs) siguiendo el principio de responsabilidad única. Segundo, se implementó un sistema de autenticación seguro basado en `bcryptjs` [?] y tokens JWT, con control de acceso basado en roles que garantiza que cada usuario solo vea la información que necesita. Tercero, se diseñó una capa de persistencia NoSQL con Firebase Firestore [?] utilizando la metodología Query Driven Design [?], y se construyó un proceso ETL [?] que migra los datos de analítica desde Firestore hacia MySQL.

La integración de estos componentes demuestra que las bases de datos NoSQL y relacionales no son excluyentes, sino complementarias: MySQL soporta la operación diaria del negocio (CRUD de productos, lotes y movimientos), mientras que Firebase almacena eventos de analítica para Big Data y procesamiento masivo. El proceso ETL conecta ambos mundos,

permitiendo que los datos de uso enriquezcan los reportes operativos.

XII-A. Posibles mejoras y líneas futuras

El equipo identificó varias áreas de mejora para futuras versiones del sistema:

- **Despliegue en la nube:** el servidor actual opera en `localhost`, lo que limita el acceso a una sola máquina. Desplegar el backend en un servicio como Render o Railway permitiría acceso desde cualquier lugar.
- **Dashboard de analítica:** crear una vista que consulte directamente Firebase para mostrar gráficos de tendencias de ventas, productos más consultados y patrones de merma en tiempo real.
- **Multi-sucursal:** activar la tabla `branch` para gestionar múltiples almacenes con inventario independiente.
- **Alertas automáticas:** implementar un *cron job* que evalúe diariamente los lotes próximos a vencer y genere registros en la tabla `alert` sin intervención manual.
- **ETL programado:** automatizar la ejecución del proceso ETL con una tarea periódica en lugar de depender del botón manual.

APÉNDICE A RECURSOS DEL PROYECTO

- **Repositorio en GitHub:**
https://github.com/KingSadid/BD_FreshStock.git
- **Código fuente del documento (L^AT_EX):**
<https://www.overleaf.com/4398971276nkbbsgfvdtb#674337>

APÉNDICE B GLOSARIO DE TÉRMINOS CLAVE

- **PWA (Progressive Web App):** aplicación web que ofrece experiencia equivalente a app nativa sin instalación desde tienda de aplicaciones.
- **PEPS:** “Primero en Entrar, Primero en Salir” (*FIFO*). Método de rotación de inventarios para productos perecederos.
- **Merma:** pérdida física de productos por caducidad, daño o deterioro.
- **Modelo ER:** diagrama que representa la estructura conceptual de una base de datos mediante entidades, atributos y relaciones.
- **KPI (Key Performance Indicator):** indicador clave de rendimiento operativo.
- **CRUD:** operaciones básicas sobre datos: *Create, Read, Update, Delete*.
- **ETL (Extract, Transform, Load):** proceso que extrae datos operativos, los transforma en información consolidada y los carga en estructuras optimizadas para analítica.
- **NoSQL:** bases de datos no relacionales, optimizadas para grandes volúmenes y estructuras de datos flexibles.

- **Query Driven Design:** metodología de modelado NoSQL que diseña la estructura a partir de las consultas más frecuentes del sistema.
- **Hashing criptográfico:** transformación irreversible de una contraseña mediante una función matemática, de modo que la contraseña original nunca se almacene en texto plano.
- **Firestore:** base de datos NoSQL en la nube ofrecida por Google Firebase, basada en colecciones de documentos con estructura JSON.
- **Big Data:** conjunto de datos tan grande y complejo que requiere herramientas especiales para su procesamiento y análisis.
- **JWT (JSON Web Token):** estándar de autenticación basado en tokens firmados que permite verificar la identidad del usuario sin almacenar sesiones en el servidor.
- **Middleware:** capa de software que se ejecuta entre la petición HTTP y la respuesta, utilizada para tareas como autenticación, logging o control de acceso.

REFERENCIAS

- [1] E. A. Silver, D. F. Pyke, and D. J. Thomas, *Inventory and Production Management in Supply Chains*, 4th ed. Boca Raton, FL, USA: CRC Press, 2016.
- [2] Project Management Institute, “Small Business Inventory Management: Best Practices,” PMI White Paper, 2019.
- [3] Food and Agriculture Organization of the United Nations, *The State of Food and Agriculture 2019: Moving Forward on Food Loss and Waste Reduction*, Rome, Italy: FAO, 2019.
- [4] Google LLC, “Material Design 3,” 2026. [Online]. Disponible en: <https://m3.material.io/>
- [5] P. A. Grassi, J. A. Fenton, E. M. Newton *et al.*, “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST Special Publication 800-63B, Gaithersburg, MD, USA, 2020.
- [6] L. Probst, F. Dameri, and J. Damiani, “Inventory Management Practices in Food Supply Chains: A Systematic Literature Review,” *Journal of Cleaner Production*, vol. 198, pp. 1345–1360, 2018.
- [7] Mozilla Developer Network, “Window.localStorage,” 2025. [Online]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [8] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Dept. Informática, Univ. of California, Irvine, 2000.
- [9] Open Worldwide Application Security Project, “Password Storage Cheat Sheet,” 2024. [Online]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [10] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.
- [11] Express.js, “Express.js Guide,” 2025. [Online]. Disponible en: <https://expressjs.com/>
- [12] N. M. Stok, “bcryptjs: Pure JavaScript implementation of bcrypt,” 2023. [Online]. Disponible en: <https://www.npmjs.com/package/bcryptjs>
- [13] K. Chodorow, *MongoDB: The Definitive Guide*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2013.
- [14] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.
- [15] Google LLC, “Cloud Firestore Documentation,” 2025. [Online]. Disponible en: <https://firebase.google.com/docs/firestore>
- [16] R. Cattell, “Scalable SQL and NoSQL Data Stores,” *ACM Computing Surveys*, vol. 39, no. 4, pp. 1–61, 2011.
- [17] L. Dejun, W. Xiaoling, and Z. Ming, “ETL Process Design for Big Data Analytics Platforms,” in *Proc. IEEE Int. Conf. on Big Data Analytics*, 2021, pp. 112–118.